



Politehnica University of Timisoara



Web Programming

C2. Client-Side Scripting



Client-Side Scripting

- Code that runs in a Web browser (client-side)
- Two ways:
 - Embedded in HTML files
 - External code files using a browser plug-in
 - Adobe Flash
 - Java Applet



JavaScript

- A lightweight programming language
- Code embedded in HTML files
- Can manipulate the HTML entities
- Interpreted by the Web Browser
- Developed by Netscape (1995). Now comply with *ECMAScript* standard (European Computer Manufacturers Association)
- Syntax similar to C and Java



ECMAScript History

Year	Name	Description
1997	ECMAScript 1	First Edition.
1998	ECMAScript 2	Editorial changes only.
1999	ECMAScript 3	Added Regular Expressions. Added try/catch.
	ECMAScript 4	Was never released.
2009	ECMAScript 5	Added "strict mode". Added JSON support.
2011	ECMAScript 5.1	Editorial changes.
2015	ECMAScript 6	Added classes and modules.
2016	ECMAScript 7	Added exponential operator (**). Added Array.prototype.includes.

- The latest version is ECMAScript 15, released in June 2024 (introduce static methods Object.groupBy and Map.groupBy and UNICODE for regular expressions etc.)



JavaScript syntax

```
public class Sample {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
        try {  
            final MissileSilo silo = new MissileSilo("silo.weapons.mil");  
            silo.launchMissile(args[0]);  
        } catch (Exception e) {  
            System.out.println("Unexpected exception: " + e);  
        }  
    }  
}
```



```
console.log('Hello world');  
try {  
    const silo = new MissileSilo('silo.weapons.mil');  
    silo.launchMissile(process.argv[0]);  
} catch(e) {  
    console.log('Unexpected exception' + e);  
}
```





JavaScript v15 features

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script>
// Create an Array
const fruits = [
  {name:"apples", quantity:300},
  {name:"bananas", quantity:500},
  {name:"oranges", quantity:200},
  {name:"kiwi", quantity:150}
];
// Callback function to select low volumes
function myCallback({ quantity }) {
  return quantity > 200 ? "ok" : "low";
}
// Group by ok and low
const result = Object.groupBy(fruits, myCallback);
// Display Results
let text = "These fruits are Ok: <br>";
for (let [x,y] of result.ok.entries()) {
  text += y.name + " " + y.quantity + "<br>";
}
text += "<br>These fruits are low: <br>";
for (let [x,y] of result.low.entries()) {
  text += y.name + " " + y.quantity + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
</body> </html>
```

These fruits are Ok:
apples 300
bananas 500

These fruits are low:
oranges 200
kiwi 150

Ref: <https://www.w3schools.com/>



JavaScript Strengths

- JS can generate text and HTML tags on the fly
- JS can dynamically change the HTML DOM entities
- Performance depends only on client computation power (very scalable)
- Useful actions (e.g., form validation)



JavaScript Weakness

- Cannot access external resources like files or databases (however, very secure)
- No protection for sensitive or copyrighted algorithms (the code is accessible to all clients – obfuscation is recommended)
- Hard to develop rich programs (good for lightweight ones) – however: OO, WebGL, and JIT compiler



Strict Mode

- Strict Mode from ECMAScript version 5
 - Disallows: undeclared variables / deleting a variable
 - Eliminates some JavaScript silent errors
 - Strict subset of the language – e.g., **with** is not permitted
 - However, it can determine variation between browsers

```
function myFunction(obj) {  
    'use strict'; // ignored by old browsers  
    var x = 17;  
    with (obj) { // !!! syntax error if ver > 5  
        x;      // var x or obj.x?  
    }  
    delete x = 17; // !!! syntax error if ver > 5  
}
```



JS Code

- Embedded in the HTML page:

```
<head>  
<script language="javascript">  
    // JavaScript code  
</script>  
</head>
```

- As external file, linked to HTML page:

```
<head>  
<script language="javascript" src="script.js">  
</script>  
</head>
```



JS Variables

- JavaScript has weak typing (loose typing)
- Variable declaration allows:
 - **Explicit syntax:** `var name = "Sam"; let name = "Sam";`
 - **Implicit syntax:** `name = "Sam";` *// not in Strict Mode*
- Variable names are case sensitive
- Allows variable redeclaration (**var**), but without losing the initial value
- Almost everything is an object (exceptions: basic types as number, string, boolean, **null**, and **undefined**). Functions are objects too!



JS Variable scope

- JavaScript has weak typing (loose typing)
- Variable declaration allows:
 - **Implicit syntax:** `name = "Sam";` // not in Strict Mode
 - **Explicit syntax:** `var name = "Sam";` `let name = "Sam";`
 - **Constants syntax:** `const name = "Sam";`
- Variable names are case sensitive
- `var` is deprecated
- `const` – values and type cannot change
- Almost everything is an object (exceptions: basic types as number, string, boolean, `null`, and `undefined`). Functions are objects too!



JS Arrays

- JS arrays are objects
- The Array object is used to store multiple heterogeneous values
- Array declarations:
 1. **var** myCars=**new Array**(); // regular declaration
myCars[0]="Dacia";
myCars[1]="Opel";
myCars[2]="BMW";
 2. **var** myCars=**new Array**("Dacia", "Opel", "BMW"); // condensed declaration
 3. **var** myCars=["Dacia", "Opel", "BMW"]; // literal declaration



JS Statements

- If...Else
- Switch
- For Loop
- For..In
- While Loop
- Break
- Continue

```
<html>
<body>
<h2>The for..in Loop</h2>
<p>A for in statement loops through the
properties of a person object:</p>
<p id="panel"></p>
<script>
const person = {fname:"John", lname:"Doe",
age:25};
let txt = "";
for (let x in person) {
  txt += (x + ": " + person[x] + "<br/>");
}
document.getElementById("panel").innerHTML = txt;
</script>
</body>
</html>
```

The for..in Loop

A for in statement loops through the properties of a person object:

fname: John
lname: Doe
age: 25



JS Functions

- A function contains code that will be executed by an event or by a call to the function
- A function may be called from anywhere within a page
- Usually, functions are declared in the `<head>` page section or as external module



JS Functions as Objects

- Functions are objects that:
 - Can be created dynamically at runtime, during the execution of the program
 - Can be assigned to variables, can have their references copied to other variables, and, except for a few special cases, can be deleted
 - Can be passed as arguments to other functions and can also be returned by other functions
 - Can have their own properties and methods!



JS Functions Declaration (I)

```
function fname(var1, var2, ..., varX)
{
    // JS code
}
```

- Parameters are passed by values for all basic data types (strings, numbers, booleans)
- Parameters are passed by reference for objects (ex. DOM objects)



JS Functions Declaration (II)

// **named** function expression

```
let add = function add(a, b) {  
    return a + b;  
};
```

```
alert(add(2, 5));    // 7
```

```
alert(add.name);    // add
```

// or **anonymous** function expression

```
let add = function (a, b) {  
    return a + b;  
};
```

```
alert(add(2, 5));    // 7
```

```
alert(add.name);    // add
```



JS Functions Arguments Array

```
function makeArray() {  
    for (i = 0; i < makeArray.arguments.length; i++)  
        this[ i ] = makeArray.arguments[ i ];  
}
```

```
var days = new makeArray(  
    'Monday', 'Tuesday', 'Wednesday',  
    'Thursday', 'Friday', 'Saturday', 'Sunday' );
```

- *Fname.arguments* – the arguments array



JS Self-Defining Functions

```
letameleon = function () {  
    alert("I'm green!");  
   ameleon = function () {  
        alert("I'm red!");  
    };  
};  
  
// using the self-defining function  
ameleon(); // I'm green!  
ameleon(); // I'm red!
```



JS Variable Scopes

- Three scopes:
 - Global
 - ~~Local to a function (not to a block) – var~~
 - Local to a block – let (cannot be redeclared)
- Antipattern (not allowed in strict mode):

```
function myFunction() {  
    var a = b = 0;    // a - local, b – global  
    // ⇔ var a = (b = 0);  
    c = 1; // c - global  
}
```



JS Variable Scopes

```
// antipattern - var is deprecated!  
myname = "global"; // global variable  
function func() {  
    // var myname; -- assumed by the interpreter (hoisting)  
    alert(myname);    // "undefined" (local var)  
    var myname = "local"; // "var" is processed before any  
                           code is executed (hoisting)  
    alert(myname);    // "local"  
}  
func();
```

- First alert point also on local variable!, considered declared but not initialized (undefined)



JS Variable Scopes

■ What is the output?

```
<script>
myFunction();
alert(carName);
alert(car2);

function myFunction() {
  let carName = "Volvo";
  car2 = "Dacia";
}
</script>
```

```
<script>
myFunction();
alert(car2);
alert(carName);

function myFunction() {
  let carName = "Volvo";
  car2 = "Dacia";
}
</script>
```



JS Events (I)

- JS allows event-driven programming
- Events are actions that can trigger some JS code
- Events are normally assigned to functions
- Events are attributes of HTML tags linked to JS code or functions



JS Events (II)

- Various events supported. Ex.:
 - onClick – link or image is clicked
 - onSubmit – a form is submitted
 - onMouseOver – the mouse moves over
 - onChange – a form control is changed
 - onLoad – entity gets loaded in the browser



JS Events Example

```
<html>
<body>
  <h1>First Event Example!</h1>
  <a href="http://www.upt.com"
    onclick="alert('Click on UPT!');
    return false">
    <img src=
      "http://www.upt.ro/images/universitatea-
      politehnica-timisoara.jpg" alt="UPT" />
  </a>
</body>
</html>
```



JS Error Handling

- Browsers implement a standard silent error handling for debugging purpose
- Error handling by **try .. catch** block

```
try {  
    // Code to check  
}  
catch(err) {  
    // Error handler  
}
```



JS Throw

- The throw statement can be used together with the try .. catch statement
- It creates an exception for a logical error
- The exception can be a string, an integer, a Boolean, or an object
- See the throw example:
(https://www.w3schools.com/js/tryit.asp?filename=tryjs_throw_error)



JS Objects (I)

- JavaScript is an Object-Oriented Programming (OOP) language, but it not necessary uses classes (available only from ECMAScript6)
- Properties are the values associated with an object (prototype-based programming)
- ‘Classes’ are special functions in JavaScript
- Methods are the actions that can be performed on objects (actually, they are proprieties too – functions are JS objects)
- Properties (and methods) can be added dynamically

Ref: <https://thewebdev.info/2020/08/05/using-classes-in-javascript/>



JS Objects (II)

- Two kinds of objects:
 - **Native**
 - Built-in (ex. **Array** or **Date**)
 - User defined
 - **Host**
 - Defined by the host environment (for example, the Web browser environment – e.g., **window** or DOM objects)
- Inheritance implemented using *prototype* object (e.g. `Object.setPrototypeOf(Cat.prototype, Animal);`)
- Inheritance implemented using *extends*



JS Objects Prototypes

- Every JavaScript object has a prototype. The prototype is also an object
- All JavaScript objects inherit their properties and methods from their prototype
- Prototype via object constructor function:

```
function person(name, age) {  
    this.name = name;  
    this.age = age;  
    this.rename = function(newname)  
        { this.name=newname; } ;  
}  
aPerson = new person("John", 24);
```



JS Obj – Private attributes

```
function Gadget() {                                // using prototype
    // private member
    let name = 'Tablet';
    // public function
    this.getName = function () {
        return name;
    };
}

var toy = new Gadget();
// 'name' is undefined, it's private
alert(toy.name); // undefined
// public method has access to 'name'
alert(toy.getName()); // "Tablet"
```




JS Obj – Private attributes

```
class Gadget {                                // using class
  constructor(name) {
    // private member
    var _name = name;
    // public function
    this.setName = function(name) { _name=name; }
    this.getName = function () { return name; }
  }
}

var toy = new Gadget('Toy');
// 'name' is undefined, it's private
alert(toy.name); // undefined
// public method has access to 'name'
alert(toy.getName()); // "Toy"
```



JS Standard Objects

- Array - https://www.w3schools.com/js/js_arrays.asp
- Boolean - https://www.w3schools.com/js/js_booleans.asp
- String - https://www.w3schools.com/js/js_strings.asp
- Date - https://www.w3schools.com/js/js_dates.asp
- Math - https://www.w3schools.com/js/js_math.asp
- RegExp - https://www.w3schools.com/js/js_regexp.asp

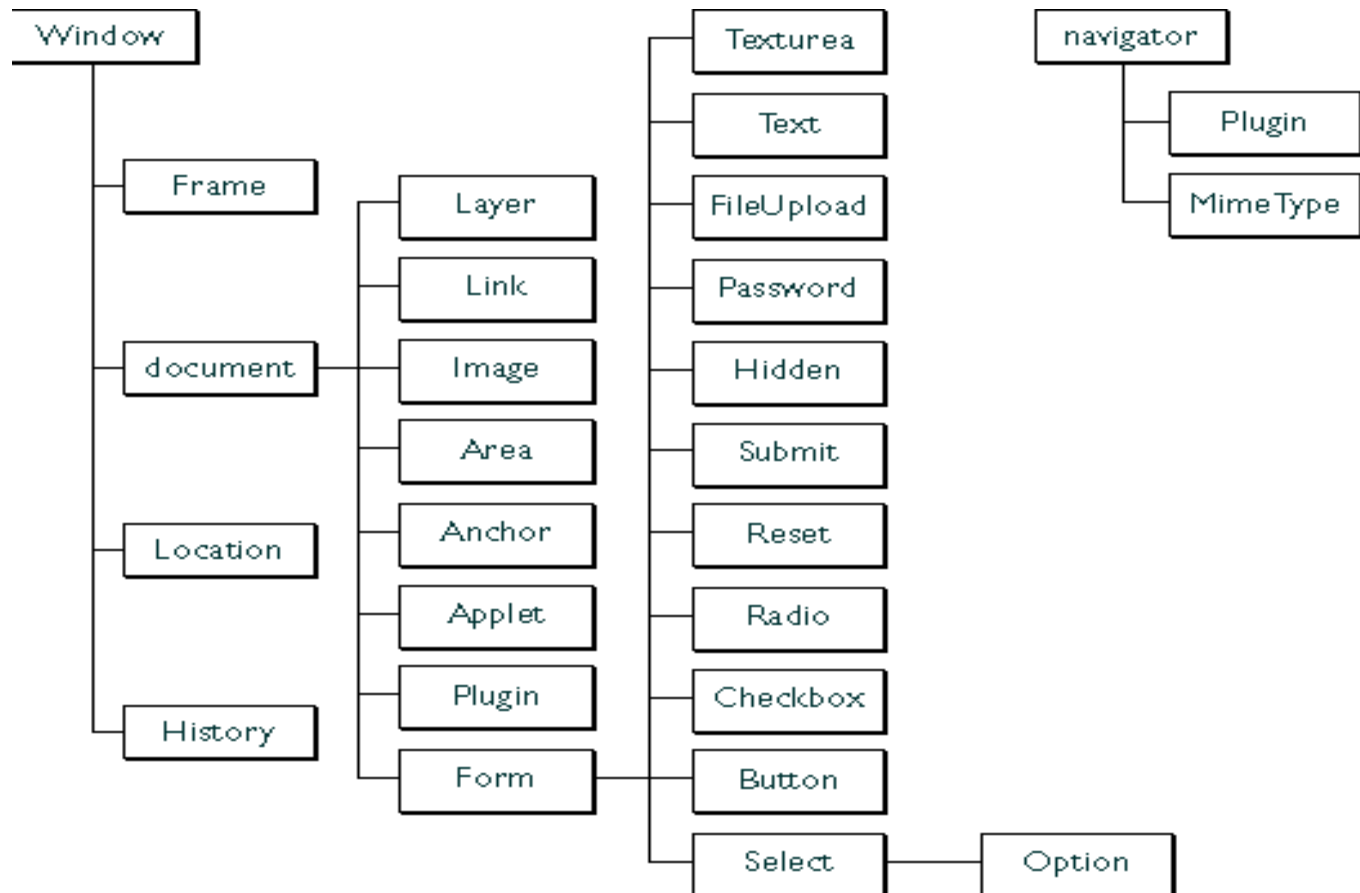


JS and DOM

- JavaScript uses the Document Object Model (DOM) to manipulate HTML
- The DOM is a hierarchy of HTML elements
- DOM uses a dot-separated syntax to refer the hierarchy levels
- DOM contains collections, properties and methods



DOM Hierarchy





JS samples (I)

- Writing page text and HTML tags:

```
document.write("<b>Some text</b>");
```

- Displaying messages

```
document.alert("Alert message!");
```

- Document closing

```
document.close();
```

- Page navigation

```
Window.location = "page.html";
```



JS samples (II)

■ Inserting text/elements in DOM:

```
<body>
  <div id = 'd1'></div> <br/>
  <div id = 'd2'></div>
  <script>
    document.getElementById("d1").innerText = '<b>abc</b> abc'
    document.getElementById("d2").innerHTML = '<b>abc</b> abc'
  </script>
</body>
```

Output:
abc abc
abc abc



JS samples (II)

- Browser Window - https://www.w3schools.com/js/js_window.asp
- JavaScript Cookies - https://www.w3schools.com/js/js_cookies.asp
- Form Validation - https://www.w3schools.com/js/js_validation.asp
- DOM Animation - https://www.w3schools.com/js/js_htmlDOM_animate.asp
- Image Maps (clickable regions) – https://www.w3schools.com/js/tryit.asp?filename=tryjs_area_coords
- Timing Events - https://www.w3schools.com/js/js_timing.asp



JS Resources

- JavaScript Tutorial

<http://www.w3schools.com/js/default.asp>

- JavaScript Examples

<https://www.programiz.com/javascript/examples>

- JavaScript Quality Control

<http://jshint.com/>

- IEEE Paper on JS advantages (September 2020)

<https://www.computer.org/publications/tech-news/build-your-career/5-reasons-javascript-is-still-better-than-python?source=byc>



JS Platforms

Ride the mobile wave



SERVVOY™

- **Servoy** is a RAD development tool and deployment platform
- **Servoy** consists of a **GUI** designer, is **event-driven** and runs **scripts** through **JavaScript**
- Has a free community edition

<http://www.servoy.com/>



VB Script

- The VBScript (Visual Basic Scripting) is a Microsoft scripting language. It is used mostly in conjunction with ASP server-side technology and works best with the *Internet Explorer* browser
- VBScript – deprecated now, used on old Ms platforms
- The VBScript code is written within paired `<script>` tags inside `<head>` and/or `<body>` section.

`<script language="VBScript">`

`<!-- VBScript code-->`

`</script>`



Adobe Action Script

- Adobe (Macromedia) ActionScript - a scripting language used mainly for the development of features in Adobe Flash applications
- Action script goes into external .swf files that run in Adobe Flash Player / browser plug-in (deprecated) and Adobe Air desktop/mobile
- It is not possible to embed ActionScript directly into a HTML page
- The current version is ActionScript 3.0